

- 5.5     Operatori u jeziku C++
  - 5.5.1     Operator pridruživanja
  - 5.5.2     Aritmetički operatori
  - 5.5.3     Operator dodjele tipa
  - 5.5.4     Logički operatori
  - 5.5.5     Poredbeni (relacijski) operatori
  - 5.5.6     Bitovni operatori
  - 5.5.7     Operator sizeof
  - 5.5.8     Operator razdvajanja
  - 5.5.9     Uvjetni operator
  - 5.5.10    Operatori za rad s pokazivačima

## 5.5 Operatori u jeziku C++

### 5.5.1 Operator pridruživanja

Operatorom pridruživanja mijenja se vrijednost nekog objekta (varijable), pri čemu tip objekta ostaje nepromijenjen. Njime se objektu na lijevoj strani pridružuje neka vrijednost s desne strane izraza.

primjer:

```
int i = 5;
i = 3;
cout << i;
```

#### 5.5.1.1 Operatori obnavljajućeg pridruživanja

Osim operatora `=`, jezik C++ za aritmetičke operatore podržava i operatore *obnavljajućeg pridruživanja* koji se sastoje od znaka odgovarajućeg aritmetičkog operatora i znaka jednakosti.

| Operatori obnavljajućeg pridruživanja |                 |                 |                 |                 |                        |                                        |
|---------------------------------------|-----------------|-----------------|-----------------|-----------------|------------------------|----------------------------------------|
| <code>+=</code>                       | <code>-=</code> | <code>*=</code> | <code>/=</code> | <code>%=</code> | <code>&gt;&gt;=</code> | <code>&lt;&lt;=</code> <code>^=</code> |

Operatori obnavljajućeg pridruživanja omogućuju kraće pisanje naredbi:  
primjer:

```
int n = 10;
n += 5;
```

Drugi red ovog primjera mogao se napisati i ovako:

```
n = n + 5;
```

### 5.5.2 Aritmetički operatori

Za ugrađene tipove podataka (`int`, `float`) definirani su osnovni aritmetički operatori poput zbrajanja, oduzimanja, množenja i dijeljenja. Ti operatori mogu se podijeliti na : *unarne*, koji djeluju samo na jedan objekt, te na binarne koji djeluju na dva objekta.

| Aritmetički operatori |                   |              |
|-----------------------|-------------------|--------------|
| unarni operatori      | <code>+X</code>   | unarni plus  |
|                       | <code>- X</code>  | unarni minus |
|                       | <code>X ++</code> | uvećaj nakon |
|                       | <code>X --</code> | umanji nakon |
|                       | <code>++X</code>  | uvećaj prije |
|                       | <code>-- X</code> | umanji prije |
|                       | <code>X+Y</code>  | zbrajanje    |
|                       | <code>X-Y</code>  | oduzimanje   |

|                   |     |            |
|-------------------|-----|------------|
| binarni operatori | X*Y | množenje   |
|                   | X/Y | dijeljenje |
|                   | X%Y | modulo     |

Osim unarnog plusa i minusa koji mjenaju predznak broja, u jeziku C++ definirani su još i unarni operatori za uvećavanje (*inkrementiranje*) i umanjivanje vrijednosti (*dekrementiranje*) broja. Operatori -- ili ++ napisani ispred objekta prvo vrše operaciju nad njim pa tek onda dohvaćaju njegovu vrijednost, a ako su napisani iza objekta redoslijed je obrnut.

primjer:

```
int i = 1;
cout << ++i;
cout << i;
cout << i--;
cout << i;
```

### 5.5.3 Operator dodjele tipa

Operatorom dodjele tipa određujemo da se sa zadanim memorijskim prostorom rukuje kao s podatkom određenog, dodijeljenog tipa.

U slijedećem primjeru potrebno je podijeliti dvije cjelobrojne varijable te rezultat pridružiti realnoj varijabli :

```
int Brojnik = 1;
int Nazivnik = 4;
float Kvocijent = Brojnik / Nazivnik;
cout << Kvocijent << endl;
```

Izvođenjem ovog primjera na ekranu će se ispisati nula. Iako je rezultat dijeljenja pridružen realnoj varijabli, to pridruživanje slijedi tek nakon što je operacija dijeljenja dva cijela broja završena, pa je prevoditelj automatski odbacio decimalna mjesta rezultata. Za dobivanje točnog rezultata potrebno je koristiti operator dodjele tipa, s njime bi naš primjer izgledao ovako:

primjer:

```
int Brojnik = 1;
int Nazivnik = 4;
float Kvocijent = (float)Brojnik / (float)Nazivnik;
cout << Kvocijent << endl;
```

Navođenjem ključnih riječi float u zagradama ispred operanda, njihove vrijednosti pretvaraju se u decimalne prije operacije dijeljenja, tako da je rezultat korektan (realnog tipa).

### 5.5.4 Logički operatori

Logički operatori koriste se za formiranje složenih logičkih izraza. C++ koristi slijedeće logičke operatore :

|                   |
|-------------------|
| Logički operatori |
|-------------------|

|        |                    |
|--------|--------------------|
| ! x    | logički komplement |
| x && y | logički I          |
| x    y | logički ILI        |

primjer:

```
enum bool {false,true};      // ako prevoditelj ne podržava tip bool
bool Istina = true;
bool Laž = false;
cout << (Istina && Laž) << endl;    // 0
cout << (Istina || Laž) << endl;    // 1
```

Razlikuju se slijedeće vrijednosti operanda : 0 - logička laž i sve vrijednosti različite od nule - logička istina. Rezultat može biti 0 - logička laž ili 1, logička istina.

### 5.5.5 Poredbeni (relacijski) operatori

Poredbeni operatori koriste se za formiranje jednostavnih logičkih izraza. Za razliku od npr. Pascala operandi moraju biti isključivo podaci jednostavnih tipova (numerički, znakovni, nabrojivi ili pokazivači). Kao rezultat usporedbe dobiva se logička vrijednost : 0-logička laž ili 1- logička istina.

| Poredbeni operatori    |                   |
|------------------------|-------------------|
| <code>x &lt; y</code>  | manje od          |
| <code>x &lt;= y</code> | manje ili jednako |
| <code>x &gt; y</code>  | veće od           |
| <code>x &gt;= y</code> | veće ili jednako  |
| <code>x == y</code>    | jednako           |
| <code>x != y</code>    | različito         |

primjer:

```
cout << (5 == 5) << endl;    // 1
cout << (5 <= 4) << endl;    // 0
```

### 5.5.6 Bitovni operatori

C++ omogućuje izravne operacije na pojedinim bitovima cjelobrojnih podataka. Na raspolaganju je šest operatora:

| Bitovni operatori         |                  |
|---------------------------|------------------|
| <code>~ i</code>          | komplement       |
| <code>i &amp; j</code>    | binarni i        |
| <code>i   j</code>        | binarni ili      |
| <code>i ^ j</code>        | isključivi ili   |
| <code>i &lt;&lt; n</code> | pomakni u lijevo |
| <code>x &gt;&gt; y</code> | pomakni u desno  |

primjer:

```
unsigned char a = 85;
unsigned char b = ~a;           // komplement od a
cout << (int)a << endl;        // 85
cout << (int)b << endl;        // 170
```

Primjer će ispisati broj 85 (01010101 binarno) i njegov bitovni komplement, broj 170 (10101010 u binarnom prikazu).

### 5.5.7 Operator sizeof

Operator *sizeof* je unarni operator koji kao rezultat daje broj bajtova što ih operand zauzima u memoriji računala:

primjer:

```
cout << "Veličina memorije za pohranjivanje podatka tipa int je : " << endl;
      << sizeof(int) << " bajtova" << endl;
```

Ispisuje se 4;

Operator *sizeof* primjenjuje se na izraze kod kojih se traži duljina rezultata. Ti izrazi mogu biti varijable, konstante, pokazivači, reference, polja, korisnički definirane klase, strukture, unije i objekti.

### 5.5.8 Operator razdvajanja

Operator razdvajanja, (zarez) koristi se za razdvajanje izraza u naredbama. Izrazi odvojeni zarezom izvode se postepeno s lijeva nadesno.

primjer:

```
int i = 10, j = 5;  
cout << i << " " << j << endl;
```

Ispisuje se : 10 5.

### 5.5.9 Uvjetni operator

Uvjetni operator omogućuje skraćeno pisanje jednostavnih selektorskih izraza. Sintaksa je slijedeća:

| Uvjetni operator          |
|---------------------------|
| uvjet ? izraz1 : izraz2 ; |

Uvjetni operator radi tako da provjerava uvjet, ako izraz uvjeta daje logičku istinu izračunava se izraz jedan, u protivnom se izračunava izraz dva.

primjer:

```
int a,x;  
if (x<0)  
    a=-x  
else  
    a=x;
```

korištenjem uvjetnog operatora ('?') može se napisati kao :

```
int a,x;  
a = (x < 0) ? -x : x;
```

Varijabli a pridružuje se -x ako je x manji od nule te x ako je x veći od nule.

### 5.5.10 Operatori za rad s pokazivačima

\* - označava pokazivač, odnosno

```
int *pok;
```

definira varijablu *pok* kao pokazivač na dinamičku varijablu tipa *int*.

& - adresni operator, označava adresu nekog podatka. Primjer :

```
int *pok;  
int staticka;
```

```
pok = &staticka;    // pridružuje pokazivačkoj varijabli adresu statičke  
                    // varijable
```

-> operator za pristup članovima struktura, unija i objekata preko pokazivača. Primjer:

```
struct tslog {  
    int mat_br;  
    char prez_i_ime[30];  
};  
tslog staticki_slog, *slog;  
slog = &staticki_slog;  
slog -> mat_br = 32000;  
strcpy (slog -> prez_i_ime, "Matić Mate");
```

new – alokacija memorijskog prostora za dinamičku varijablu. Primjer :

```
int *pok;  
pok = new int;
```

rezervira se memorijski prostor za dinamičku varijablu tipa *int* i usmjerava pokazivač *pok* na nju.

delete – dealokacija memorijskog prostora na koji pokazuje pokazivač. Primjer :

```
int *pok;  
pok = new int; // alokacija memorijskog prostora za dinamičku varijablu  
...  
delete pok;    // dealokacija memorijskog prostora za dinamičku varijablu,
```